LINQ

Ghid Complet LINQ - Sintaxă Declarativă (Query Syntax)

Cuprins

1. Selecție și Filtrare
2. Sortare
3. Grupare
4. Join-uri
5. Operații de Agregare
6. Operații de Set
7. Selecție Avansată
8. Operații pe Colecții Nested
9. Paginare și Limitare
10. Conversii
11. Exemple Combinate
12. Șabloane Frecvente

---

1. Selecție și Filtrare

`from` - Definește sursa de date

```
from g in games
```

`where` - Filtrează elemente

```
from g in games
where g.ReleaseYear > 2020
select g
```

`select` - Proiectează rezultate

```
// Selectează obiecte întregi
from g in games
select g

// Selectează doar un câmp
```

```
from g in games
select g.Name

// Selectează obiecte anonime
from g in games
select new { g.Name, g.ReleaseYear }
```

---

2. Sortare

`orderby` - Sortare ascendentă

```
from g in games
orderby g.ReleaseYear
select g
```

`orderby ... descending` - Sortare descendentă

```
from g in games
orderby g.NumberOfQuests descending
select g
```

Sortare multiplă

```
from g in games
orderby g.Producer, g.ReleaseYear descending
select g
```

---

3. Grupare

`group ... by` - Grupează elemente

```
// Grupare simplă
from g in games
group g by g.Producer

// Grupare cu selectare
from g in games
group g by g.Producer into producerGroup
select new {
    Producer = producerGroup.Key,
    Count = producerGroup.Count(),
```

```
    Games = producerGroup.ToList()
}
```

---

4. Join-uri

`join` - Inner join

```
from g in games
join p in producers on g.Producer equals p.Name
select new { g.Name, ProducerCountry = p.Country }
```

`join ... into` - Group join (Left outer join)

```
from p in producers
join g in games on p.Name equals g.Producer into gameGroup
select new {
    Producer = p.Name,
    Games = gameGroup.ToList()
}
```

---

5. Operații de Agregare

După `select`, folosești metode de extensie:

Count - Numără elemente

```
(from g in games select g).Count()
```

Sum - Sumează valori

```
(from g in games select g.NumberOfQuests).Sum()
```

Average - Calculează media

```
(from g in games select g.NumberOfOpponents).Average()
```

Min - Găsește minimul

```
(from g in games select g.ReleaseYear).Min()
```

Max - Găsește maximul

```
(from g in games select g.NumberOfQuests).Max()
```

First/FirstOrDefault - Primul element

```
(from g in games orderby g.ReleaseYear select g).FirstOrDefault()
```

Last/LastOrDefault - Ultimul element

```
(from g in games orderby g.ReleaseYear select g).LastOrDefault()
```

Single/SingleOrDefault - Un singur element

```
// Aruncă excepție dacă sunt mai multe elemente
(from g in games where g.Name == "Skyrim" select g).SingleOrDefault()
```

Any - Verifică dacă există elemente

```
(from g in games where g.ReleaseYear > 2020 select g).Any()
```

All - Verifică dacă toate elementele îndeplinesc condiția

```
(from g in games select g).All(g => g.ReleaseYear < 2025)
```

---

6. Operații de Set

Distinct - Elimină duplicate

```
(from g in games select g.Producer).Distinct()
```

Union - Reuniune

```
var query1 = from g in games1 select g;
var query2 = from g in games2 select g;
var union = query1.Union(query2);
```

Intersect - Intersecție

```
var intersect = query1.Intersect(query2);
```

Except - Diferență

```
var except = query1.Except(query2);
```

---

7. Selecție Avansată

`let` - Definește variabile temporare

```
from g in games
let totalContent = g.NumberOfQuests + g.NumberOfOpponents
where totalContent > 100
select new { g.Name, totalContent }
```

`into` - Continuare query

```
from g in games
where g.ReleaseYear > 2015
select g into recentGames
where recentGames.NumberOfQuests > 50
select recentGames
```

---

8. Operații pe Colecții Nested

`SelectMany` (în sintaxă declarativă)

```
// Dacă ai o listă de producători cu liste de jocuri
from producer in producers
from game in producer.Games
select game
```

---

9. Paginare și Limitare

Take - la primele N elemente

```
(from g in games select g).Take(5)
```

Skip - Sare peste primele N elemente

```
(from g in games select g).Skip(3)
```

TakeLast - la ultimele N elemente

```
(from g in games select g).TakeLast(2)
```

SkipLast - Sare peste ultimele N elemente

```
(from g in games select g).SkipLast(1)
```

TakeWhile - la elemente cât timp condiția e adevărată

```
(from g in games orderby g.ReleaseYear select g).TakeWhile(g =>
g.ReleaseYear < 2020)
```

SkipWhile - Sare peste elemente cât timp condiția e adevărată

```
(from g in games orderby g.ReleaseYear select g).SkipWhile(g =>
g.ReleaseYear < 2015)
```

---

10. Conversii

ToList() - Convertește la listă

```
var list = (from g in games select g).ToList()
```

ToArray() - Convertește la array

```
var array = (from g in games select g).ToArray()
```

ToDictionary() - Convertește la dicționar

```
var dict = (from g in games select g).ToDictionary(g => g.Name)
```

ToLookup() - Creează lookup (dicționar cu valori multiple)

```
var lookup = (from g in games select g).ToLookup(g => g.Producer)
```

---

11. Exemple Combinate pentru Examen

Exemplu 1: Filtrare + Sortare + Selecție

```
var result = from g in games
             where g.ReleaseYear > 2015
             orderby g.NumberOfQuests descending
             select new { g.Name, g.NumberOfQuests };
```

Exemplu 2: Grupare + Agregare

```
var result = from g in games
             group g by g.Producer into producerGroup
             select new {
                 Producer = producerGroup.Key,
                 TotalQuests = producerGroup.Sum(g => g.NumberOfQuests),
                 AvgOpponents = producerGroup.Average(g =>
g.NumberOfOpponents),
                 GameCount = producerGroup.Count()
             };
```

Exemplu 3: Let + Where + Select

```
var result = from g in games
             let score = g.NumberOfQuests * 2 + g.NumberOfOpponents
             where score > 200
             orderby score descending
             select new { g.Name, Score = score };
```

Exemplu 4: Grupare complexă

```
var result = from g in games
             group g by g.ReleaseYear into yearGroup
             where yearGroup.Count() > 1
             orderby yearGroup.Key
             select new {
                 Year = yearGroup.Key,
                 Games = yearGroup.Select(g => g.Name).ToList(),
                 TotalQuests = yearGroup.Sum(g => g.NumberOfQuests)
             };
```

Exemplu 5: Join + Grupare

```
var result = from g in games
             join p in producers on g.Producer equals p.Name
             group g by p.Country into countryGroup
             select new {
                 Country = countryGroup.Key,
                 GameCount = countryGroup.Count(),
                 TotalQuests = countryGroup.Sum(g => g.NumberOfQuests)
             };
```

Exemplu 6: Multiple Where + Let

```
var result = from g in games
             where g.ReleaseYear > 2010
             let quality = g.NumberOfQuests * 0.6 + g.NumberOfOpponents *
0.4
             where quality > 50
             orderby quality descending
             select new { g.Name, Quality = quality };
```

---

12. Șabloane Frecvente la Examen

1. Găsește maximul unui câmp

```
var max = (from x in items
           orderby x.Field descending
           select x).FirstOrDefault();

// SAU
var maxValue = (from x in items select x.Field).Max();
```

2. Media pentru elemente filtrate

```
var avg = (from x in items
           where x.Condition > value
           select x.Field).Average();
```

3. Grupare cu count

```
var groups = from x in items
             group x by x.Category into g
             select new {
```

```
                Category = g.Key,
                Count = g.Count()
            };
```

4. Top N elemente

```
var topN = (from x in items
            orderby x.Score descending
            select x).Take(5);
```

5. Verificare existență

```
bool exists = (from x in items
               where x.Name == "Value"
               select x).Any();
```

6. Găsește minimul cu filtrare

```
var min = (from x in items
           where x.Category == "SomeCategory"
           orderby x.Value
           select x).FirstOrDefault();
```

7. Sumă totală pe grup

```
var totalByGroup = from x in items
                   group x by x.Category into g
                   select new {
                       Category = g.Key,
                       Total = g.Sum(x => x.Value)
                   };
```

8. Găsește elemente cu valoare peste medie

```
var average = (from x in items select x.Value).Average();
var aboveAverage = from x in items
                   where x.Value > average
                   select x;
```

9. Count cu condiție multiplă

```
var count = (from x in items
             where x.Year > 2020 && x.Category == "Tech"
```

```
                select x).Count();
```

10. Selecție distinctă după proprietate

```
var distinctCategories = (from x in items
                          select x.Category).Distinct();
```

Operatori de Comparație în WHERE

```
// Egalitate
where x.Name == "Value"

// Inegalitate
where x.Value != 100

// Mai mare / Mai mic
where x.Year > 2020
where x.Year < 2025

// Mai mare sau egal / Mai mic sau egal
where x.Score >= 50
where x.Score <= 100

// Și logic (AND)
where x.Year > 2020 && x.Category == "Tech"

// Sau logic (OR)
where x.Year < 2010 || x.Year > 2020

// Negare
where !(x.IsDeleted)

// Contains (pentru strings)
where x.Name.Contains("Game")

// StartsWith / EndsWith
where x.Name.StartsWith("The")
where x.Name.EndsWith("3")

// În interval
where x.Year >= 2010 && x.Year <= 2020
```

Structura Generală a unui Query LINQ

```
var result = from [element] in [collection]
             [join ...]
             [let ...]
             [where ...]
             [orderby ... (descending)]
             [group ... by ... into ...]
             select [projection]
             [.Method()];
```

Ordinea de Execuție Recomandată

1. `from` - definește sursa
2. `join` - combină cu alte surse (dacă e necesar)
3. `let` - definește variabile calculate (dacă e necesar)
4. `where` - filtrează elementele
5. `orderby` - sortează (dacă e necesar)
6. `group by` - grupează (dacă e necesar)
7. `select` - proiectează rezultatul final
8. Metode de extensie (`.ToList()` , `.Average()` , etc.)

Sfaturi pentru Examen

✅ **Întotdeauna** pune `.ToList()` , `.ToArray()` sau `.FirstOrDefault()` la final pentru a materializa query-ul

✅ Pentru agregări (`Average` , `Sum` , `Count` , `Max` , `Min`) poți apela direct pe rezultatul query-ului

✅ `group by` necesită aproape întotdeauna `into` pentru a lucra cu grupurile

✅ `orderby descending` pentru sortare descendentă

✅ Combină `where` cu operatori: `>` , `<` , `==` , `!=` , `&&` , `||`

✅ Atenție la sintaxă: `on field1 equals field2` (nu `==`)

✅ Folosește obiecte anonime cu `new { Property = value }` pentru proiecții

✅ `producerGroup.Key` accesează cheia grupării (valoarea după care s-a grupat)

✅ În `group by` , tot ce vrei să folosești după grupare trebuie accesat prin `g.Property` sau prin metode de agregare

✅ Pentru debug, întotdeauna verifică dacă query-ul returnează `null` sau colecție goală

Greșeli Frecvente de Evitat

❌ **NU** folosi `==` în join, folosește `equals`

```
// GREȘIT
join p in producers on g.Producer == p.Name

// CORECT
join p in producers on g.Producer equals p.Name
```

❌ **NU** uita `.ToList()` sau `.FirstOrDefault()` când vrei să materializezi query-ul

```
// Poate cauza probleme
var result = from g in games select g;

// CORECT
var result = (from g in games select g).ToList();
```

❌ **NU** uita `into` după `group by` dacă vrei să folosești grupul

```
// GREȘIT - nu poți accesa grupul
from g in games
group g by g.Producer
select new { Producer = ???, Count = ??? }

// CORECT
from g in games
group g by g.Producer into producerGroup
select new { Producer = producerGroup.Key, Count = producerGroup.Count() }
```

❌ **NU** încerca să accesezi proprietăți după grupare fără agregare

```
// GREȘIT
from g in games
group g by g.Producer into producerGroup
select new { producerGroup.Name } // Name nu există pe grup!

// CORECT
from g in games
group g by g.Producer into producerGroup
select new { Producer = producerGroup.Key, Names = producerGroup.Select(g =>
g.Name).ToList() }
```

Exemple Practice Complete

Exemplu 1: Jocuri PC

```
List<PC_Game> games = new List<PC_Game>
{
    new PC_Game("The Witcher 3", "CD Projekt Red", 2015, 100, 50),
    new PC_Game("Cyberpunk 2077", "CD Projekt Red", 2020, 50, 100),
    new PC_Game("Skyrim", "Bethesda", 2011, 200, 75),
};

// Toate jocurile lansate după 2015
var recent = from g in games
             where g.ReleaseYear > 2015
             select g;

// Jocul cu cele mai multe quest-uri
var mostQuests = (from g in games
                  orderby g.NumberOfQuests descending
                  select g).FirstOrDefault();

// Media opponents pentru jocuri cu peste 50 quests
var avgOpponents = (from g in games
                    where g.NumberOfQuests > 50
                    select g.NumberOfOpponents).Average();

// Grupare după producer
var byProducer = from g in games
                 group g by g.Producer into producerGroup
                 select new {
                     Producer = producerGroup.Key,
                     Count = producerGroup.Count(),
                     TotalQuests = producerGroup.Sum(g => g.NumberOfQuests)
                 };
```

Exemplu 2: Articole Științifice

```
List<ScientificArticle> articles = new List<ScientificArticle>
{
    new ScientificArticle("AI Research", "John Doe", 2021, 50, 3.5),
    new ScientificArticle("Quantum Computing", "Alice Smith", 2020, 40,
4.0),
    new ScientificArticle("Climate Change", "Bob Brown", 2019, 35, 2.8),
};

// Articole după 2019
```

```
var recent = from a in articles
             where a.YearOfPublication > 2019
             select a;

// Articolul cu cele mai multe pagini
var longest = (from a in articles
               orderby a.NumberOfPages descending
               select a).FirstOrDefault();

// Media paginilor pentru articole după 2020
var avgPages = (from a in articles
                where a.YearOfPublication > 2020
                select a.NumberOfPages).Average();

// Articole cu impact factor peste medie
var avgImpact = (from a in articles select a.ImpactFactor).Average();
var highImpact = from a in articles
                 where a.ImpactFactor > avgImpact
                 orderby a.ImpactFactor descending
                 select new { a.Title, a.ImpactFactor };
```

Resurse Adiționale

- **Microsoft Docs LINQ**: https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/
- **LINQ Query Syntax vs Method Syntax**: Ambele sunt echivalente, dar la examen se cere sintaxa declarativă (Query Syntax)
- **Debugging**: Folosește `.ToList()` și breakpoint-uri pentru a vedea rezultatele intermediate

---

Succes la examen! 🚀

Creat pentru examenul la Concepte Fundamentale ale Limbajelor de Programare